

【赛前蓄能】PAC 2026 初赛必看：鲲鹏 KUPL 库与底层调优基础知识全解析！

摘要：PAC 2026 优化赛道开赛在即！为了帮助参赛选手提前熟悉高性能计算平台的底层技术生态，本期为大家带来官方公开推荐的底层加速库——鲲鹏 KUPL 库的基本概念，以及实用的性能采集与绑核基础指南。快叫上你的队友一起提前充电吧！

引言

在高性能计算与代码性能优化中，除了算法层面的改进，对特定硬件架构特性的极致压榨往往是决定胜负的关键。在 PAC 2026 的优化赛道中，如何高效利用计算平台提供的高性能基础库和硬件调优工具，是每位选手的必修课。为了让大家在开赛后能够快速上手，今天我们就来盘点一下鲲鹏计算生态中两款重磅的公开工具——统一并行加速库与轻量级性能采集库 libkperf，并聊聊多核架构下必知必会的 NUMA 绑核基础知识。

一、认识核心引擎：鲲鹏统一并行加速库 (KUPL)

在面对密集型计算或大规模数据搬移任务时，手写底层汇编或 Intrinsics 固然能精细控制，但开发周期长、容错率低。鲲鹏统一并行加速库正是为了解决这一痛点而设计的公共基础库。它针对鲲鹏架构进行了深度的底层硬件级优化，主要包含以下核心基础模块：

1. mma 矩阵乘加模板库

- 基本概念：mma (Matrix Multiply-Accumulate) 是针对密集矩阵运算设计的通用模板库。
- 技术优势：它深入底层封装了硬件级的矩阵乘加指令，并提供了高度优化的分块与流水线控制模板。掌握这个库的调用，可以帮助选手在处理通用密集计算任务时，快速替代传统的手写 Micro-kernel，提升开发效率与执行性能。

2. memcpy 数据搬移模块

- 基本概念：高性能计算中，内存带宽往往是第一大瓶颈。处理非连续内存访问、高维数据重排时，传统的标准库函数往往难以跑满带宽。
- 技术优势：memcpy 提供了针对鲲鹏架构优化的硬件级数据异步搬移与对齐加速。熟练运用该模块，能够有效降低数据准备和数据交换阶段的访存延迟。

📖 开源基础学习直通车：

- 讲解视频：华为鲲鹏社区 KUPL 课程[链接](#)
- 示例代码：openEuler Git 仓 (kupil-sample)[链接](#)
- 实现源码：KunpengCompute GitCode 仓[链接](#)

二、性能调优的“透视眼”：libkperf 采集库

在优化代码的过程中，如果只用 time 命令看一个总时间，犹如盲人摸象。我们需要知道 CPU 到底是在高速运转，还是在痛苦地“空转等待”。

鲲鹏架构提供了轻量级的 PMU 采集库 —— libkperf，它是选手进行算子分析时的强力辅助。

1. 为什么要了解 libkperf?

通过 libkperf，你可以精准捕捉代码在运行过程中的底层硬件事件：

- Cache Miss（缓存未命中造成的访存延迟）
- Branch Misprediction（分支预测失败带来的流水线白费）
- Instruction Stall（指令流水线停顿原因）

通过这些定量的硬件计数，你就能精准定位性能瓶颈究竟是在“算力不足”还是在“访存被卡”。

2. C/C++ API 通用调用框架

在编写基准测试时，可以通过在代码中嵌入 libkperf API 来测量特定计算代码段的硬件表现：

C++

```
1 #include "kperf_pmu.h"
2 #include <stdio.h>
3
4 int main() {
5     // 1. 初始化并指定要采集的公开 PMU 事件
6     char *pmu_events[] = {"hw:cycles", "hw:instructions"};
7     int pd = PmuOpen(COUNT_TYPE_RAW, pmu_events, 2, NULL, 0);
8
9     if (pd < 0) {
10         printf("PMU Open Failed!\n");
11         return -1;
12     }
13
14     // 2. 开启采集
15     PmuEnable(pd);
16
17     // =====
18     // 执行需要测试的计算任务
19     // RunYourComputationTask();
20     // =====
21
22     // 3. 关闭采集
```

```

23     PmuDisable(pd);
24
25     // 4. 读取并打印硬件计数结果
26     PmuData *data = NULL;
27     int len = PmuRead(pd, &data);
28     for (int i = 0; i < len; i++) {
29         printf("Event: %s, Count: %lld\n", data[i].eventName, data[i].count);
30     }
31
32     PmuClose(pd);
33     return 0;
34 }
35

```

开源基础学习直通车：

- 代码仓主页：[openeuler/libkperf](#)
- 详细使用文档：[Details_Usage.md](#)
- C/C++ API 说明：[C_C++_API.md](#)

三、SME 技术Blog

来自华为官方的SME技术学习[Blog](#)

实战避坑小贴士

在很多高性能计算集群的日常运维管理中，为了保证系统服务的稳定运行，通常会在每个 NUMA 节点的最后一个物理核设置核隔离，将其专门留给系统后台服务。

- 避坑指南：大家在日常编写绑核脚本或在代码中使用 `pthread_setaffinity_np` 进行线程绑定时，要养成主动避开这些系统隔离核的习惯。如果把密集的计算任务强制绑定到隔离核上，会因为系统服务的频繁打扰，导致程序执行时间出现剧烈抖动，无法跑出稳定的极致性能。

推荐学习资源：

- 绑核、绑 NUMA 实用基础教程：清华大学 HPC 团队绑核指南[链接](#)

四、究极进化

PAC大赛前期已经上线“PAC大讲堂”，我们为参赛人员提供了究极进化教程：

- 平台使用教程[链接](#)
- 鲲鹏统一并行框架介绍与开发指导[链接](#)

- 毕昇编译器介绍及调优指南[链接](#)

五、总结

工欲善其事，必先利其器。在 PAC 2026 优化赛道正式打响前，提前克隆上述公开的加速库与工具链代码仓，在本地或公共环境里跑通几个简单的示例，能帮你省去开赛初期大量的环境摸索时间。

祝大家在即将到来的比赛中取得理想的成绩！

欢迎在评论区交流多核优化心得，点赞、在看、转发，助你的战队赛前战力翻倍！